



AMERICAN
UNIVERSITY
OF BEIRUT

AI-Assisted Coding

Module 2 Additional Readings

From Idea to Backend Implementation with AI

Purpose and Use

This resource supports Module 2 only. Module 2 is not a general FastAPI course and it is not a production-deployment module. It is about using an editor-based AI tool to turn the Module 1 FastAPI skeleton into a verified Task Tracker backend: strict Pydantic v2 models, in-memory storage, five CRUD endpoints, status-transition business rules, and a pytest suite proven by a Break Test.

Core Readings:

Read only the relevant sections. The purpose is to reinforce the lecture, not to become a full documentation study session.

Core Reading 1 — Editor-based AI prompting and file context

What to focus on: File/context references, specific prompts, review before applying, and using the editor chat as a coding partner rather than a replacement for verification.

Skip for this module: Autonomous agents, MCP, CLI, enterprise setup, and advanced team configuration.

Links: [Cursor Docs: Prompting agents](#) | [GitHub Copilot best practices](#) | [VS Code Chat overview](#)

Core Reading 2 — FastAPI request and response fundamentals

What to focus on: Map the docs to the Task Tracker API: TaskCreate is the request body, TaskResponse is the response model, and status_code belongs on the route decorator.

Skip for this module: Advanced response classes, custom encoders, dependency injection, and security features.

Links: [FastAPI Request Body](#) | [FastAPI Response Model](#) | [FastAPI Response Status Code](#)

Core Reading 3 — FastAPI paths, query parameters, and error handling

What to focus on: Path vs query parameters; returning 200 with an empty list for no matches; raising HTTPException for 404 or business-rule failure.

Skip for this module: Custom exception handlers and global error-handling architecture.

Links: [FastAPI Path Parameters](#) | [FastAPI Query Parameters](#) | [FastAPI Handling Errors](#)

Core Reading 4 — Pydantic v2 models, validators, configuration, and enums

What to focus on: Recognizing correct generated code, especially rejecting blank titles, rejecting unknown fields, using enums, and avoiding Pydantic v1 patterns.

Skip for this module: Advanced nested models, custom serializers, settings management, and generic models.

Links: [Pydantic Models](#) | [Pydantic Validators](#) | [Pydantic Model Config](#) | [Pydantic Serialization / model_dump\(\)](#)

Core Reading 5 — PATCH and partial updates

What to focus on: exclude_unset behavior, partial updates, and the difference between validating a field value and validating a state transition.

Skip for this module: Database update patterns and ORM-specific update code.

Links: [FastAPI Body Updates / PATCH](#) | [Pydantic Serialization / model_dump\(\)](#)

Core Reading 6 — Testing FastAPI with pytest and fixtures

What to focus on: TestClient setup, pytest fixture basics, autouse fixtures, status-code assertions, and why tests must be proven with a Break Test.

Skip for this module: AsyncClient, mocking, parametrized fixtures, plugins, and integration testing with real databases.

Links: [FastAPI Testing](#) | [pytest fixtures](#)

Practical References:

These are lookup references. Keep them nearby while coding or reviewing AI output; do not read them end to end.

- **Editor AI context:** Use when deciding which files to attach before prompting. [Cursor prompting](#) | [Copilot Chat in IDE](#)
- **HTTP status codes:** Use when checking 201, 204, 404, and 422 expectations. [FastAPI status code](#) | [MDN HTTP status codes](#) | [MDN 204 No Content](#) | [MDN 422 Unprocessable Content](#)
- **Pydantic v2 verification:** Use when checking field_validator, extra="forbid", enums, and model_dump(). [Validators](#) | [Model Config](#) | [Serialization](#)
- **CRUD endpoint review:** Use when checking request body, response_model, path params, query params, and HTTPException. [Request Body](#) | [Response Model](#) | [Path Parameters](#) | [Query Parameters](#) | [Handling Errors](#)
- **Test suite review:** Use when checking tests/conftest.py, TestClient, created_task fixture, and autouse storage reset. [FastAPI Testing](#) | [pytest fixtures](#)

Optional Deeper Dives: For Students Who Want More Detail

- Microsoft Learn — Introduction to prompt engineering with GitHub Copilot: a structured, beginner-friendly prompt practice module. Optional because it is broader than the Task Tracker implementation.
Links: [Open Microsoft Learn prompt engineering module](#)
- Microsoft Learn — Developer use cases for AI with GitHub Copilot: a broader look at Copilot across the software-development lifecycle. Optional because Module 2 focuses only on editor-based backend implementation.
Links: [Open Microsoft Learn developer use cases module](#)
- GitHub Blog — Using Copilot in your IDE: practical tips for adding context in an editor. Optional because product details can change and the recordings use Cursor as the demonstrated tool.
Links: [Open GitHub Blog article](#)
- Martin Fowler — Exploring GenAI: useful for professional reflection on AI-assisted delivery. Optional because it is not a coding reference for the Task Tracker API.
Links: [Open Martin Fowler series](#)